LESSON 1

1

if と for をもう一度

目標：if 文と for 文が「どう動くか」を説明できるようになる

if 文の復習

if 文：条件で「やること」を選ぶ

C# Program.cs

```
1  string text = Console.ReadLine();
2  int hp = int.Parse(text);
3  if (hp ⩽ 0)
4  {
5      Console.WriteLine("Game Over");
6  }
7  else
8  {
9      Console.WriteLine("Continue");
10 }
```

hp ⩽ 0 ？

true（はい）
Game Over

false（いいえ）
Continue

- ( ) の中が true → if の { } を実行
- false → else の { } を実行
- どちらか **片方だけ** が必ず実行される

if 文の復習

条件の書き方：比較と「かつ」「または」

記号	意味	例
==	等しい	hp == 0
!=	等しくない	hp != 0
< >	より小さい / 大きい	score > 80
<= >=	以下 / 以上	score >= 60
&&	かつ（両方 true）	a > 0 && b > 0
||	または（どちらか true）	a == 0 || b == 0

「60 以上 80 未満」の書き方

✕ 書けない

```
60 ≤ score < 80
```

◯ && で 2 つに分ける

```
score ≥ 60 && score < 80
```

注意 以上・以下は = が入る（>= / <=）、より大きい・未満は = が入らない（> / <）。問題文の言葉をよく見よう。

if 文の復習

else if は「上から順に」1 つだけ実行される

×順番がまちがい（score = 90）

C# Program.cs

```
1  if (score ⩾ 60)
2  {
3      Console.WriteLine("合格");
4  }
5  else if (score ⩾ 80)
6  {
7      Console.WriteLine("優秀");
8  }
```

出力

合格　← 優秀 が出ない！

○ きびしい条件を先に（score = 90）

C# Program.cs

```
1  if (score ⩾ 80)
2  {
3      Console.WriteLine("優秀");
4  }
5  else if (score ⩾ 60)
6  {
7      Console.WriteLine("合格");
8  }
```

出力

優秀

POINT　最初に true になった所だけ実行して、残りは見ない。範囲がせまい（きびしい）条件から先に書く。

for 文の復習

for 文の 3 つの部品

```
for (int i = 0; i < 3; i++)
```

① `int i = 0`

準備（最初に 1 回だけ）

数える用の変数 i を作って 0 を入れる

② `i < 3`

条件（毎周の最初に確認）

true なら中身を実行。false になったら for を終わる

③ `i++`

更新（1 周の最後）

i を 1 増やして、② に戻る

POINT 動く順番：① → ② → 中身 → ③ → ② → 中身 → ③ → … → ② が false で終了。i は 0, 1, 2 と変わり、3 で終わる。

for 文の復習

「N 回くり返す」の書き方

書き方	i の値	回数	使いどころ
for (int i = 0; i < n; i++)	0, 1, …, n-1	n 回	基本の形。配列の番号と同じなので相性◎
for (int i = 1; i <= n; i++)	1, 2, …, n	n 回	「1 から N まで」の数そのものを使いたいとき

- **配列を全部見るときは i < 配列名.Length**
- numbers[numbers.Length] は存在しない（範囲外エラー）

C# Program.cs

```
1  for (int i = 0; i < numbers.Length; i++)
2  {
3      Console.WriteLine(numbers[i]);
4  }
```

POINT 迷ったら「int i = 0; i < 回数; i++」。0 から始めて < で止める、と覚えよう。

動きを追う道具

トレース表：1 周ずつ紙に書いて追う

C# Program.cs

```
1  int total = 0;
2  for (int i = 1; i ⩽ 4; i++)
3  {
4      total += i;
5  }
6  Console.WriteLine(total);
```

出力

```
10
```

周	i	i <= 4	total（中身のあと）
1 周目	1	true	0 + 1 = 1
2 周目	2	true	1 + 2 = 3
3 周目	3	true	3 + 3 = 6
4 周目	4	true	6 + 4 = 10
終了	5	false	（for を抜ける）

POINT 頭の中でやらずに必ず紙に書く。「変数ごとに 1 列」「1 周ごとに 1 行」。組み合わせで迷ったら、まずトレース。

LESSON 2

組み合わせの「型」

2

目標：よく出る 7 つの型と、3 つの置き場所を使いこなす

LESSON 2

このコマの目標

- コードを「for の前・中・後」の 3 つの置き場所に分けて考えられる
- 問題文のことばから、使う「型」を選べる
- 7 つの型を、何も見ずに書けるようになる

いちばん大事な考え方

3 つの置き場所：for の「前・中・後」

① for の前：1 回だけ

準備。入れ物を作る

count = 0 / total = 0 / isFound = false

② for の中：N 回

1 つずつ調べる。if で判定して、数える・足す・更新する

③ for の後：1 回だけ

結果を出す。出力 / Yes・No の判定

C# Program.cs

```
1  // ① for の前：準備
2  int count = 0;
3  for (int i = 0; i < n; i++)
4  {
5      // ② for の中：1 つずつ調べる
6      if (条件)
7      {
8          count++;
9      }
10 }
11 // ③ for の後：結果を出す
12 Console.WriteLine(count);
```

POINT 迷ったら「この行は何回実行したい？」と考える。1 回 → for の前か後　／　N 回 → for の中。

問題文の読み方

問題文のことばが「型」を教えてくれる

問題文のことば	使う型	使う道具
「それぞれ」「すべて」「N 個の〜を」	型① 全部に同じことをする	for
「〜なら」「〜だけ」「〜のものを」	型② 条件に合うものだけ	for ＋ if
「何個」「何人」「何回」	型③ 数える	for ＋ if ＋ count
「合計」「全部でいくつ」	型④ 合計する	for ＋ total
「一番大きい」「最高」「最小」	型⑤ 最大・最小	for ＋ if ＋ max
「含まれているか」「あるか」「Yes / No」	型⑥ あるかないか	for ＋ if ＋ bool
「最初の」「何番目にあるか」	型⑦ 見つけたら止める	for ＋ if ＋ break

POINT 問題文を読んだら、まずこのことばに線を引こう。線を引いたことばの数だけ「型」を組み合わせる。

組み合わせの型

型①　全部に同じことをする

それぞれ　すべて

- **for だけ。if はいらない**
- 配列の要素を 1 つずつ取り出して、同じ処理をする

入力例

```
3 5 8
```

出力

```
6
10
16
```

C# Program.cs

```csharp
// numbers：スペース区切りの数値を受け取った配列
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i] * 2);
}
```

POINT　すべての型の土台。ここに if や変数を足していくと、ほかの型になる。

組み合わせの型

型②　条件に合うものだけ

〜なら　〜だけ　〜のもの

- **for の中に if を入れる**
- if で「選ぶ」。合わないものは何もしない
- else はいらないことが多い

入力例

```
3 4 7 10
```

出力

```
4
10
```

C# Program.cs

```
1  // numbers：スペース区切りの数値を受け取った配列
2  for (int i = 0; i < numbers.Length; i++)
3  {
4      if (numbers[i] % 2 == 0)
5      {
6          Console.WriteLine(numbers[i]);
7      }
8  }
```

POINT　「偶数だけ」「60 点以上だけ」「HP が 0 の敵だけ」…条件を変えれば何にでも使える。

組み合わせの型

型③　数える（カウント）

何個　何人　何回

- 前：count = 0 を用意
- 中：条件に合ったら count++
- 後：count を出力

入力例

```
5
75
40
60
90
55
```

出力

```
3
```

C# Program.cs

```
1  string text = Console.ReadLine();
2  int n = int.Parse(text);
3  const int PASS_LINE = 60;
4  int passCount = 0;
5  for (int i = 0; i < n; i++)
6  {
7      string line = Console.ReadLine();
8      int score = int.Parse(line);
9      if (score ⩾ PASS_LINE)
10     {
11         passCount++;
12     }
13 }
14 Console.WriteLine(passCount);
```

① for の前　② for の中　③ for の後

組み合わせの型

型④　合計する（＋型②との組み合わせ）

合計　全部で

全部の合計

C# Program.cs

```
1  int total = 0;
2  for (int i = 0; i < numbers.Length; i++)
3  {
4      total += numbers[i];
5  }
6  Console.WriteLine(total);
```

入力 3 8 5 6 → 出力

22

合計　〜だけ

奇数だけの合計（型② ＋ 型④）

C# Program.cs

```
1  int oddTotal = 0;
2  for (int i = 0; i < numbers.Length; i++)
3  {
4      if (numbers[i] % 2 == 1)
5      {
6          oddTotal += numbers[i];
7      }
8  }
9  Console.WriteLine(oddTotal);
```

入力 3 8 5 6 → 出力

8

① for の前　② for の中　③ for の後

組み合わせの型

型⑤　最大・最小を見つける

一番大きい　最高　最小

- 前：max に最初の値を入れる
- 中：今の max より大きければ入れかえ
- 後：max を出力
- 最小なら > を < にするだけ

入力例

4 9 2 7

出力

9

C# Program.cs

```
1  // numbers：スペース区切りの数値を受け取った配列
2  int max = numbers[0];
3  for (int i = 1; i < numbers.Length; i++)
4  {
5      if (numbers[i] > max)
6      {
7          max = numbers[i];
8      }
9  }
10 Console.WriteLine(max);
```

① for の前　② for の中　③ for の後

POINT　「チャンピオンに挑戦者が 1 人ずつ挑む」イメージ。勝ったら（大きければ）チャンピオン交代。

組み合わせの型

型⑥　あるかないか（フラグ）

含まれているか　Yes / No

- 前：isFound = false（まだ見つかってない）
- 中：見つけたら true にする
- 後：isFound を見て Yes / No

入力例

```
5
1 5 3
```

出力

```
Yes
```

C# Program.cs

```
1  // target：探す数、numbers：数値の配列
2  bool isFound = false;
3  for (int i = 0; i < numbers.Length; i++)
4  {
5      if (numbers[i] == target)
6      {
7          isFound = true;
8      }
9  }
10 if (isFound)
11 {
12     Console.WriteLine("Yes");
13 }
14 else
15 {
16     Console.WriteLine("No");
17 }
```

① for の前　② for の中　③ for の後

組み合わせの型

型⑦　見つけたら止める（break）

最初の　何番目

- **見つけたら break で for をすぐ抜ける**
- 前：見つからなかったとき用に -1 を入れておく
- 「何番目」は i + 1（i は 0 から）

入力例

```
7
3 7 5 7
```

出力

```
2
```

C# Program.cs

```
1  // target：探す数、numbers：数値の配列
2  int position = -1;
3  for (int i = 0; i < numbers.Length; i++)
4  {
5      if (numbers[i] == target)
6      {
7          position = i + 1;
8          break;
9      }
10 }
11 Console.WriteLine(position);
```

① for の前　② for の中　③ for の後

POINT　break がないと最後に見つけた 7（4 番目）が答えになってしまう。「最初の」なら break。

よくある間違い①

置き場所がずれている

✕出力を for の中に書いた

C# Program.cs

```
1  int total = 0;
2  for (int i = 0; i < numbers.Length; i++)
3  {
4      total += numbers[i];
5      Console.WriteLine(total);
6  }
```

入力 3 5 8 → 出力

3
8
16　← 途中経過も出てしまう

出力は 1 回だけ → **for の後**へ

✕count = 0 を for の中に書いた

C# Program.cs

```
1  for (int i = 0; i < numbers.Length; i++)
2  {
3      int count = 0;
4      if (numbers[i] > 5)
5      {
6          count++;
7      }
8  }
9  Console.WriteLine(count);
```

結果

エラー（count が見つからない）

毎周 0 に戻るうえ、for の外では使えない → **for の前**へ

よくある間違い②

for の中で Yes / No を決めてしまう

✕ for の中に else

C# Program.cs

```
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] == target)
    {
        Console.WriteLine("Yes");
    }
    else
    {
        Console.WriteLine("No");
    }
}
```

入力 target=5、numbers=1 5 3 → 出力

```
No
Yes
No
```

- **1 つ見るたびに Yes / No を出してしまう**
- 「ない」と言えるのは、**全部見終わってから**
- 途中で決められるのは「あった」だけ
- → 型⑥ フラグを使う：中では true にするだけ、Yes / No は for の後

注意 「for の中の else」を書いたら一度止まって考えよう。本当に毎回やりたいことか？

一覧

組み合わせの型 一覧（演習中に見返そう）

型	① for の前	② for の中	③ for の後
① 全部に同じこと	—	処理する	—
② 条件に合うものだけ	—	if (条件) { 処理 }	—
③ 数える	count = 0	if (条件) { count++ }	count を出力
④ 合計する	total = 0	total += 値	total を出力
⑤ 最大・最小	max = 最初の値	if (値 > max) { max = 値 }	max を出力
⑥ あるかないか	isFound = false	if (条件) { isFound = true }	if (isFound) で Yes / No
⑦ 見つけたら止める	position = -1	if (条件) { 記録; break; }	position を出力

POINT 型はくっつけて使える。例：「60 点以上の人数と合計」→ 型② の if の中に、型③ と 型④ を両方書く。

LESSON 3

問題を「型」に分けて解く

3

目標：問題文 → 日本語の手順 → 型 → コード の順で解けるようになる

例題 STEP 1

型を１つだけ使う問題

数列の値を全部＊２して出力 型① 全部に同じこと

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_multi/edit?language_uid=c-sharp

数列の和 型④ 合計する

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_sum/edit?language_uid=c-sharp

数列の最大値 型⑤ 最大

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_max/edit?language_uid=c-sharp

数列の最小値 型⑤ 最小

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_min/edit?language_uid=c-sharp

POINT 解く前に「使う型」と「for の前・中・後に何を書くか」を紙に書いてから、コードを書き始めよう。

例題 STEP 2

for の中に if を入れる問題

数列の中に何個ある？ 型③ 数える

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_count/edit?language_uid=c-sharp

数列の何番目にある？ 型⑦ 見つけたら止める

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_i-th/edit?language_uid=c-sharp

0 が含まれていないか判定 型⑥ あるかないか

https://paiza.jp/works/mondai/conditions_branch/conditions_branch__simple_boss/edit?language_uid=c-sharp

注意

「0 が含まれていないか判定」は、0 が**含まれていない**ときに YES。フラグが true / false のどちらのときに YES を出すか、よく読もう。

例題 STEP 3

型を組み合わせる問題

数列の A 番目から B 番目までの和 型④ ＋ for の範囲

https://paiza.jp/works/mondai/loop_problems/loop_problems__seq_partsum/edit?language_uid=c-sharp

FizzBuzz 型② ＋ else if の順番

https://paiza.jp/works/mondai/loop_problems/loop_problems__fizzbuzz/edit?language_uid=c-sharp

POINT A 番目〜B 番目は、配列の番号だと A - 1 〜 B - 1。FizzBuzz は「3 と 5 の両方で割り切れる」をいちばん先に書く。

POINT 早く終わった人は、LESSON 2 の型を見ずに STEP 1・2 の問題をもう一度書いてみよう。

授業課題（兼 宿題）

スキルチェックに挑戦

- 授業内で取り組み、時間内に終わらなかった問題は宿題とします

C170：目標ポイント
https://paiza.jp/challenges/852/ready

C169：ポイント還元セール
https://paiza.jp/challenges/846/ready

C160：降水量の計測
https://paiza.jp/challenges/792/ready

POINT スキルチェックは自分の力で解くこと。解く前に問題文のことばに線を引き、使う「型」を書き出してから始めよう。